CMPSC 448 / MIDTERM PROJECT REPORT

Who Wrote It? Identifying LLMs from Their Responses

A CNN and LSTM comparison using the WildBench V2 model-output dataset

Afolami Elabanjo | October 11, 2026

Summary

I tested whether neural text classifiers can distinguish responses generated by GPT-4o, Claude 3 Sonnet, and Gemini 1.5 Pro. I used 979 prompts with one response from each model, giving 2,937 labeled examples. All responses to a given prompt stayed in the same data split. I trained a convolutional neural network (CNN) and a long short-term memory network (LSTM) on prompt-only, response-only, and combined inputs. The response-only CNN performed best among the six neural-network experiments, with 67.7% held-out accuracy and 67.5% macro-F1. The LSTM reached 42.9% accuracy on responses. Neither model exceeded the 33.3% chance baseline using prompts alone.

2,937	67.7%	42.9%
979 matched prompts	CNN, response only	LSTM, response only

Questions investigated

RQ1. Can the generating LLM be identified from its response? I compare the CNN and LSTM using output text only.

RQ2. Does adding the user prompt help? I compare prompt-only, response-only, and prompt-plus-response classification.

RQ4 (exploratory). Which measurable writing features differ across models? I examine response length, formatting, and a simple feature-based baseline.

Scope: the NBA comparison responses already in the repository were exploratory examples. The quantitative benchmark reported here uses WildBench V2 and is not restricted to basketball prompts.

1. Dataset and collection

The data comes from AllenAI's publicly released *WildBench V2 Model Outputs* dataset. I selected one snapshot from each of three LLM families. The script **build_wildbench_data.py** downloads the model-output records and pairs responses using their session identifiers. It checks that the prompt text is identical across families, discards records without usable prompt or response text, and retains only prompts with all three outputs.

Family	Source model configuration	Matched responses
GPT	gpt-4o-2024-05-13	979
Claude	claude-3-sonnet-20240229	979
Gemini	gemini-1.5-pro	979

This process produced **2,937 responses from 979 distinct prompts**. The dataset includes math, coding, reasoning, writing, information-seeking, and other tasks. The original rows are preserved in the generated CSV with a model label, prompt, response, source-model name, session ID, and task tag. The primary CSV used for training contains the required fields **LLM_name**, **LLM_Input**, and **LLM_output**.

Data partition and leakage checks

Split	Prompts	Responses	Responses per model
Training	685	2,055	685
Validation	98	294	98
Held-out test	196	588	196
Total	979	2,937	979

I used group-aware random splitting with seed 448: approximately 70% training, 10% validation, and 20% testing. The grouping variable was the prompt itself. This is necessary because the same question was answered by all three LLMs: if a prompt appeared in both training and test sets, the model could exploit repeated input wording. The script checks for overlapping prompts; the test runner checks for unknown labels and runs five unit tests before training.

What the preprocessing does

Missing records and duplicate response rows are removed. Labels are normalized to GPT, Claude, and Gemini. During model training, Keras **TextVectorization** learns its vocabulary from the training data and converts texts into fixed-length token sequences. The held-out test texts are not used to fit the vocabulary or select stopping epochs.

All three classes are exactly balanced in every split. The project can be rebuilt from the public dataset rather than depending on copied or manually reconstructed model outputs.

2. Model design and training

I used the same basic text-processing setup for both required neural models, changing the text encoder. This makes the six experiments straightforward to compare. Both models classify one of three source labels with a softmax output layer.

Setting	Value / method
Input variants	Prompt only; response only; prompt + separator + response
Vocabulary	At most 20,000 learned tokens
Maximum input length	300 tokens, padded or truncated
Embedding layer	96 dimensions
CNN encoder	96 one-dimensional filters; kernel size 5; ReLU; global max pooling
LSTM encoder	64 recurrent units
Regularization	Dropout = 0.35 after the text encoder
Final layer	3-class softmax
Objective / optimizer	Sparse categorical cross-entropy / Adam
Training	Batch size 32; at most 8 epochs in the GitHub run
Stopping / randomness	Early stop on validation loss, patience 3; seed 448

Controlled comparison

Each model is fitted separately for three views of the input. With **prompt only**, the classifier never sees the answer. With **response only**, it sees the answer but not the prompt. With **combined input**, the prompt is placed before a separator and the response. The labels, split membership, vocabulary budget, maximum sequence length, and evaluation metrics are otherwise the same.

The vocabulary is adapted to training text for each experiment. The CNN uses convolution to detect local token patterns and max pooling to collect them. The LSTM reads the token sequence recurrently. Both feed their encoded representation to a dropout layer and a three-class softmax classifier.

How performance is measured

Accuracy is the share of correctly classified test responses. Macro-F1 is the average of the three class-level F1 scores and gives Claude, Gemini, and GPT equal weight. I also report a confusion matrix, because the same overall accuracy can hide poor recall for one model. With three equally represented classes, a constant-label classifier would score 33.3% accuracy.

These are results from one random seed and one training run per configuration. The report does not treat them as averages over repeated trials.

3. Results: source-model classification

The CNN on response text alone was the strongest of the required neural-network models. It classified **398 of 588** held-out responses correctly. The response-only LSTM reached 42.9%. Both response-based models beat the 33.3% balanced-class baseline, although the difference between their results was large.

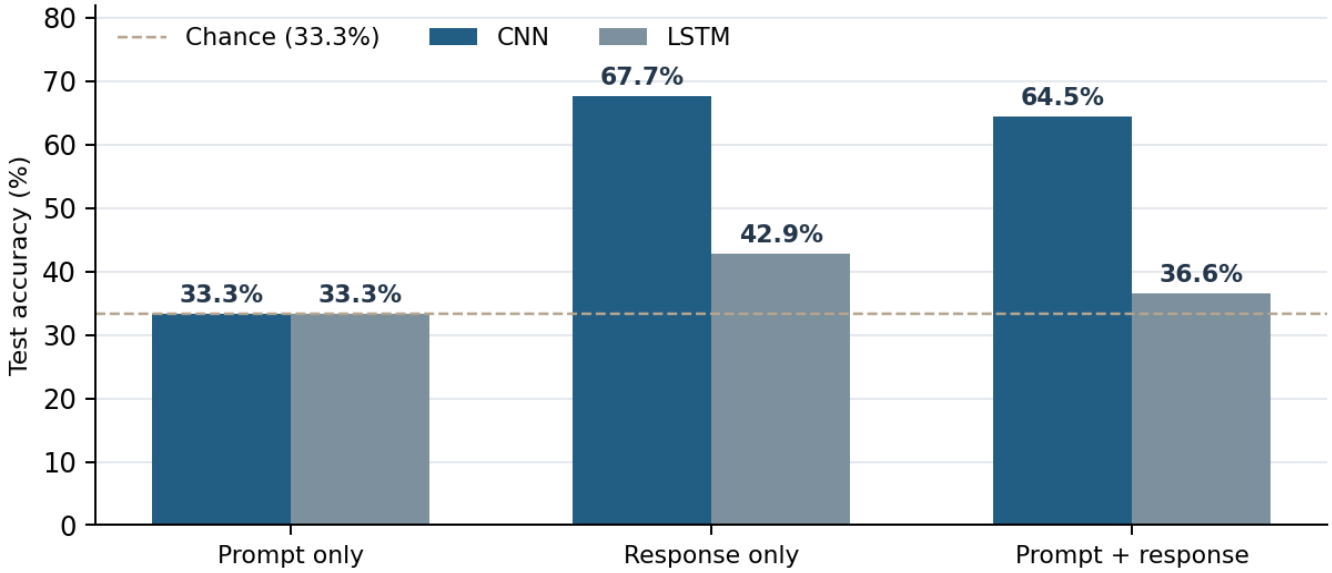


Figure 1. Held-out accuracy for the six required experiments. The dashed line marks the 33.3% equal-class baseline.

Text supplied to classifier	CNN accuracy	CNN macro-F1	LSTM accuracy	LSTM macro-F1
Prompt only	33.3%	30.7%	33.3%	17.9%
Response only	67.7%	67.5%	42.9%	41.0%
Prompt + response	64.5%	64.5%	36.6%	26.9%

RQ1: Can responses identify the model?

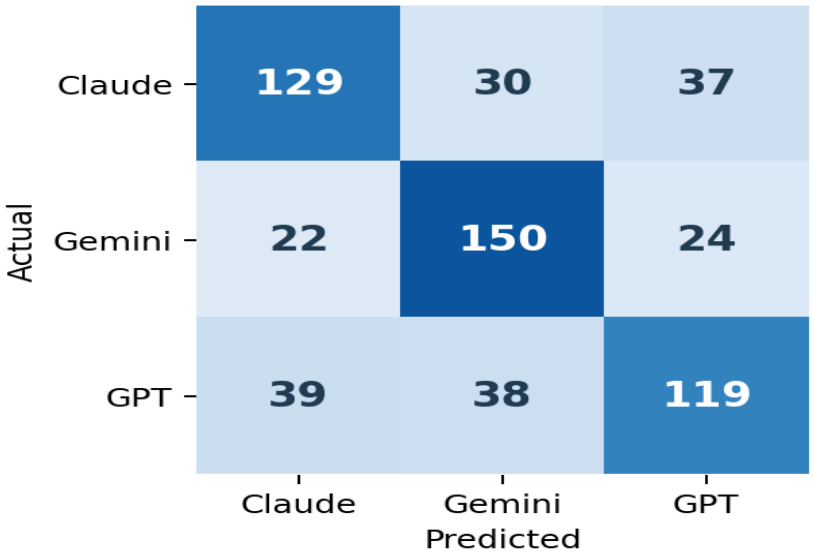
For this dataset, yes, to a useful but imperfect degree. The CNN separated the three model families with about twice the accuracy of chance. The LSTM also found some source-related signal, but it made substantially more errors. The outcome supports a narrower conclusion: writing patterns in these particular model versions and tasks can help identify the generating family. It does not establish reliable authorship detection for unseen models or later versions.

RQ2: Does including the prompt help?

No improvement appeared in these experiments. Both prompt-only models scored exactly one-third accuracy, as expected because each test prompt appears once per family with the same text. Adding the prompt before the response reduced CNN accuracy from 67.7% to 64.5%, and LSTM accuracy from 42.9% to 36.6%. The combined setup may lose useful answer tokens when the 300-token limit is reached; this is a plausible explanation, not a separately tested cause.

4. Error analysis

The strongest neural model was the CNN trained on responses only. Its confusion matrix shows how its errors were distributed across the three families. Every true class contained 196 test examples.



Class	Recall
Claude	65.8%
Gemini	76.5%
GPT	60.7%

Figure 2. Response-only CNN confusion matrix. Rows are true source models; columns are predicted models. Diagonal cells are correct.

Gemini responses were easiest for this CNN to identify: it correctly labeled 150 of 196. Claude had 129 correct and GPT had 119 correct. When the CNN predicted the wrong family, Claude and GPT were commonly confused with each other (37 Claude responses predicted as GPT, and 39 GPT responses predicted as Claude). This confusion matrix is more informative than reporting accuracy alone.

What failed in the LSTM

The LSTM was less consistent across the input variants. In the combined setting it labeled **174 of 196** true GPT responses correctly, but only **4 of 196** Gemini responses. That gave it 36.6% overall accuracy with a much lower 26.9% macro-F1. Its predictions favored GPT heavily, so the apparently above-chance accuracy should not be read as balanced success across all classes.

Possible effect of truncation

The input sequence is capped at 300 tokens. In the collected data, about **18.8% of distinct prompts** contain at least 300 whitespace-separated words before any answer text is appended. Keras tokenization can count tokens differently, but this is enough to flag a design limitation: for some combined examples, the model may see little or none of the answer. A follow-up test with a longer limit or answer-first ordering would help isolate that effect.

5. RQ4: writing and formatting signals

I compared basic properties of all 979 responses per model, then trained a small logistic-regression baseline on the same training and test partitions. These checks are exploratory: they can identify useful measurable differences, but they do not prove why a neural model made a particular prediction.

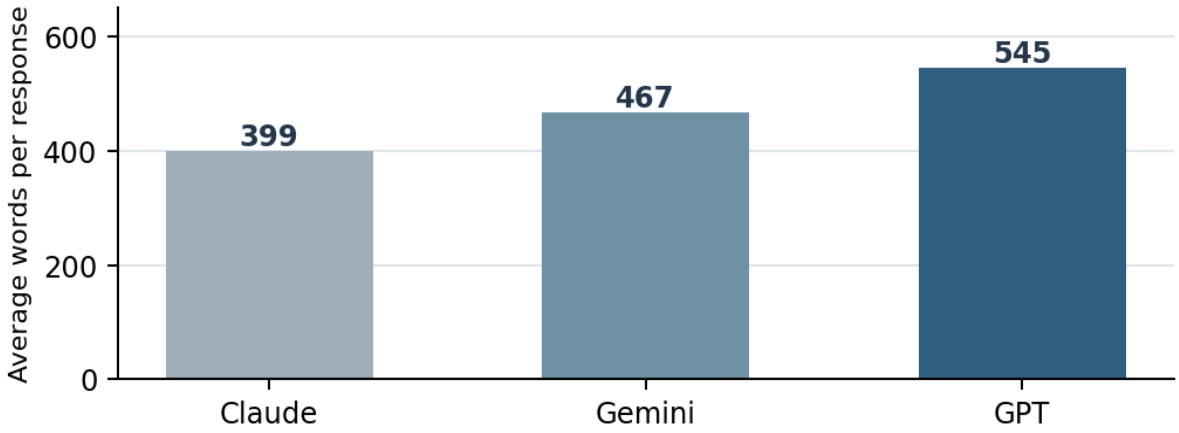


Figure 3. Mean response length across the matched prompt set, measured by whitespace-separated words.

Model	Mean words	Bulleted list	Markdown heading	Bold marker
Claude	399	26.3%	5.4%	8.1%
Gemini	467	73.5%	36.8%	86.3%
GPT	545	49.8%	54.6%	71.3%

GPT had the longest average responses, while Claude had the shortest. Gemini used markdown bold markers especially often and had the highest share of responses with hyphen or asterisk bullets. GPT had the highest share with markdown headings. These are observable patterns in this collected output, not general rules for every release of those models.

Simple baseline trained on the same split

Features	Test accuracy	Macro-F1
Log(response word count) only	43.9%	41.0%
Length + six structural features	72.3%	71.6%

The seven-feature model uses log word count, bullet-list presence, numbered-list presence, markdown-heading presence, code-fence presence, bold-marker presence, and log line-break count. A standardized multinomial logistic regression was fitted on the training split only, then evaluated on the same 588 held-out responses. Even these simple features classified authorship better than chance; the full structural baseline slightly exceeded the response-only CNN. This suggests that formatting and length deserve closer investigation. The comparison is not architecture-controlled, because the neural models read at most 300 tokens and the feature baseline uses summary measures of the complete responses.

6. Limitations and lessons

Limitations

Dataset scope. These outputs come from one public benchmark and one selected version of each model family. The benchmark spans multiple tasks, but the evaluation does not test whether the same patterns hold for different generations of GPT, Claude, or Gemini. The NBA examples in the repository were not included in the numerical benchmark.

One split and one seed. All reported neural results come from one grouped split and one training run each. Repeating the runs across several seeds could show whether the CNN/LSTM gap is stable.

Representation limits. The 300-token cap reduces the amount of a long response available to the CNN or LSTM. This especially matters in the combined setting because the prompt is placed first. The input-only result is a useful leakage check but does not mean that no dataset could ever reveal a model from a prompt.

Structural artifacts. The RQ4 baseline can exploit formatting conventions present in these downloaded records. Some conventions may reflect the way responses were requested or stored as much as intrinsic model behavior. A formatting-removal ablation would be a useful next experiment.

Lessons and experience

The most useful lesson for me was how much the data split determines whether a result is believable. Since every question has three model answers, ordinary row-level splitting would have put identical prompt text on both sides of the test boundary. Grouping by prompt made the prompt-only comparison meaningful. I also saw why accuracy alone is insufficient: the combined LSTM leaned heavily toward GPT, which was clearer in its confusion matrix and macro-F1.

Building the test pipeline also required debugging two practical issues. The repository already contained a file named **data**, so the scripts could not create a folder with that name; moving generated files to **generated/** fixed the path conflict. Keras also rejected NumPy Unicode arrays during fitting; passing text as TensorFlow string datasets resolved the training error. After those fixes, the GitHub workflow completed all six experiments and uploaded the result files.

Reproducing the benchmark

From the repository root, install **requirements.txt**, run **build_wildbench_data.py** to create **generated/llm_responses.csv**, run **prepare_test_data.py** for the grouped split, then run **evaluate_models.py** against **generated/splits**. The GitHub Actions workflow automates this sequence. It stores test predictions, class-level metrics, confusion matrices, and the comparison table as the **cmpsc448-test-results** artifact. The exploratory feature baseline in this review was computed from those same split files.

References and project materials

[1] AllenAI. *WildBench V2 Model Outputs* (public model-output dataset).
https://huggingface.co/datasets/allenai/WildBench-V2-Model-Outputs

[2] Afolami Elabanjo. *CMPSC 448 Midterm 1 GitHub repository* (code, dataset builder, and workflow).
github.com/afolamie/CMPSC-448-Midterm-1-NBA-Stats-1947-present-

[3] *Who Wrote It? Identifying LLMs from Their Responses*. CMPSC 448 assignment sheet, revised September 17, 2026.

Evaluation date: October 11, 2026. All reported CNN/LSTM measurements are from the successful GitHub Actions run 38167743136. The separate RQ4 logistic-regression comparison was calculated from the same saved training and test data.